py-cidr

Release 4.0.0

Gene C

Jul 27, 2026

CONTENTS

CHAPTER

ONE

PY-CIDR

1.1 Overview

py-cidr : python module providing network / CIDR tools

1.2 Key features

- Built on python's native ipaddress module
- 3 Classes : Cidr, CidrMap, CidrFile
- Cidr provides for many common operations for example:
  - Support for IPv4 and IPv6
  - compact lists of CIDRs to smallest set of CIDR blocks
  - convert an IP range to a list of CIDRs
  - Identify and validate
  - many more
- CidrFile offers common operations on files with lists of cidrs.
  - Includes atomic file writes
- CidrMap provides a class that maps CIDRs to values.
  - File cache employs locking to ensure multiple processes handle cache correctly.

See API reference documentation for more details.

1.3 New / Interesting

4.0.0

- CidrMap: - use Patricia26 (instead of PyTricia) for the Patricia trees. - maps are now very much smaller, about 1/6 th the size of previous files.

  A benefit that Patricia26 stores only the necessary data.
  - cache files are automatically converted to the new format.
  - PrefixMap: compact defaults to False.

More information here for patricia26.

```
py-cidr-cache-print <cache_directory>
```

1.4 Documentation:

We include pre-built versions of both html and PDF documentation, including the API reference.

The PDF file is *Docs/py-cidr.pdf* and after the package is installed it will be available:

PDF Documentation.

and a browser can be used to view:

HTML Documentation.

CHAPTER

TWO

GETTING STARTED

All git tags are signed with arch@sapience.com key which is available via WKD or download from https://www.sapience.com/tech. Add the key to your package builder gpg keyring. The key is included in the Arch package and the source= line with *?signed* at the end can be used to verify the git tag. You can also manually verify the signature

2.1 py-cidr module

2.1.1 module functions

The library provides the following tools:

CidrMap Class

CidrMap provides an optimized tool to map(network-prefix) to a value. The map may be saved to file and reused.

To make use of file cache, a *cache_directory* must be provided when instantiating the class. Note that there are 2 typed of maps, non-compact and compact. The default is non-compact. With this every (prefix, value) is added to the map.

For a compact map, effort is made to reduce redundant entries. For example if the map contains the (prefix, tuple) = ('10.0.0.0/22', 'net-1') and then a new tuple ('10.0.0.0/24', 'net-1') is added to the map, the new one is considered redundent since the network is a subnet of *'10.0.0.0/22'* and the value, *'net-1'* is the same. If the value was different, then it is not considered redundent.

For non-compact maps, every (prefix, value) is added.

A *CidrMap* contains 2 separate maps. A *PrefixMap* for IPv4 and one for IPv6.

Ths will create both an IPv4 and an IPv6 cache file in the given directory. The code is careful about reading and writing the cache files and uses locking and atomic writes to ensure the integrity of the data.

For example if application starts, reads cache, updates with new items and some time later saves the cache - the module will detect if the cache changed (by another process using same cache directory) since it was last read in, and merge its own changes with the changes in the cache file before writing out the updated cache. This should ensure nothing gets lost.

This was built this originally for our firewall tool, where part of the data gathering component creates maps of network prefixes (blocks of IP addresses) to geolocated country codes and other useful information such as the ASN(s) the prefix belongs to.

When looking up a CIDR, there are 2 methods available. The first, *lookup_lmp()* returns the (lmp_prefix, val) pair where *lmp_prefix* is the network with the longest matching cidr prefix. The longer the cidr prefix, the more specific the network. A */24* os more specific than a */22* for example.

The method *lookup_all()* returns every matching prefix and its associated value. the LMP is always the first element in returned in the list.

Since parallelizing often provides decent speedups, *CidrMap* provides a mechanism to do that. It allows each separate process or thread to work with private (thread local) cache. Each of the private data caches can then be merged together by the top level process or thread.

This avoids multiple threads/processes writing to the same in memory data at the same time. This is done using the *CidrMap::merge()* method.

Additional details are available in the API reference documentation.

Methods provided:

- CidrMap.add_prefix_val()
- CidrMap.add_prefix_vals()
- CidrMap.lookup_lmp()
- CidrMap.lookup_all()
- CidrMap.items()
- CidrMap.save_cache()
- CidrMap.merge()

Static functions: * CidrMap.create_private_cache()

Cidr Class

See the API reference in the documentation for details. This class provides a suite of tools we found ourselves using often, so we encapsulated them in this class. All methods in the class are *@staticmethod* and thus no instance of the class is needed. Just call them as you would any function (*Cidr.xxx()*).

IP addresses and networks can be represented as strings or the format used by python's native *ip_address* module. Most functionality is available on both.

Here is a sample of some of the available functionality, see the API doc for complete documentation.

- Cidr.is_valid_ip4()
- Cidr.is_valid_ip6()
- Cidr.is_valid_cidr()
- Cidr.ip_version()
- Cidr.cidr_iptype()
- Cidr.cidr_to_net
- Cidr.cidrs_to_nets
- Cidr.net_to_cidr
- Cidr.nets_to_cidrs
- Cidr.ip_to_address
- Cidr.ips_to_addresses
- Cidr.address_to_ip
- Cidr.addresses_to_ips
- Cidr.cidr_set_prefix
- Cidr.cidr_is_subnet
- Cidr.compact_cidrs

- Cidr.compact_nets
- Cidr.cidrs2_minus_cidrs1
- Cidr.sort_cidrs
- Cidr.get_host_bits
- Cidr.clean_cidr
- Cidr.fix_cidr_host_bits
- Cidr.range_to_cidrs
- Cidr.cidr_range_split
- Cidr.rfc_1918_cidrs
- Cidr.is_rfc_1918

CidrFile Class

This class provides a few reader/writer tools for files with lists of CIDR strings. Readers ignores comments. All methods are *@staticmethod* and thus no instance of the class is required. Simply use them as functions (*Cidr.xxx()*)

- Cidr.read_cidr_file(file:str, verb:bool=False) -> [str]:
- Cidr.read_cidr_files(targ_dir:str, file_list:[str]) -> [str]
- Cidr.write_cidr_file(cidrs:[str], pathname:str) -> bool
- Cidr.read_cidrs(fname:str|None, verb:bool=False) -> (ipv4:[str], ipv6:[str]):
- Cidr.copy_cidr_file(src_file:str, dst_file:str) -> None

CHAPTER

THREE

APPENDIX

3.1 Installation

Available on * Github * Archlinux AUR

On Arch you can build using the provided PKGBUILD in the packaging directory or from the AUR. To build manually, clone the repo and :

```
rm -f dist/*
/usr/bin/python -m build --wheel --no-isolation
root_dest="/"
./scripts/do-install $root_dest
```

When running as non-root then set root_dest a user writable directory

3.2 Dependencies

Run Time :

- python (3.13 or later)
- lockmgr

Building Package :

- git
- hatch (aka python-hatch)
- wheel (aka python-wheel)
- build (aka python-build)
- installer (aka python-installer)
- rsync

Optional for building docs :

- sphinx
- python-myst-parser
- python-sphinx-autoapi
- texlive-latexextra (archlinux packaguing of texlive tools)

Building docs is not really needed since pre-built docs are provided in the git repo.

3.3 Philosophy

We follow the *live at head commit* philosophy as recommended by Google's Abseil team[1]. This means we recommend using the latest commit on git master branch.

3.4 License

[1] https://abseil.io/about/philosophy#upgrade-support

CHAPTER

FOUR

LICENSE

Python module that provides IP / network / CIDR tools

CHAPTER

FIVE

API REFERENCE

This page contains auto-generated API reference documentation[1].

5.1 py_cidr

Public Methods for py_cidr module

5.1.1 Submodules

py_cidr.cidr_class

Class providing some common CIDR utilities

Module Contents

class Cidr

Provides suite of CIDR tools.

All mathods are (static) and are thus called without need to instantiate the class. For example:

net = Cidr.cidr_to_net(cidr_string)

Notation:

- cidr means a string
- net means ipaddress network (IPv4Network or IPv6Network)
- ip means an IP address string
- addr means an ip address (IPv4Address or IPv6Address)
- address means either a IP address or a cidr network as a string

static address_iptype(*addr: py_cidr._network.cidr_types.IPvxAddress | py_cidr._network.cidr_types.IPvxNetwork*) → str

Identify address or net (IPvxNetwork) as ipv4, ipv6 or neither.

Args:
addr (str): ipaddress IP or network .

Returns:
str | None: 'ip4', 'ip6' or ''

[1] Created with sphinx-autoapi

static address_to_net(*addr: py_cidr._network.cidr_types.IPAddress | py_cidr._network.cidr_types.IPvxNetwork*, *strict: bool = False*) → py_cidr._network.cidr_types.IPvxNetwork | None

Convert an address to IPvxNetwork.

Be flexible with input address. Can be IPAddress (includes string) or even IPvxNetwork.

Args:

addr (IPAddress | IPvxNetwork):
Input address.

strict (bool):
If true then cidr is considered invalid if host bits are set. Defaults to False. (see ipaddress docs).

Returns:

IPvxNetwork | None:
The IPvxNetwork derived from input "addr" or None if not an address/network.

static addresses_to_ips(*addresses: list[py_cidr._network.cidr_types.IPvxAddress]*) → list[str]

From list of IPs in ipaddress format, get list of ip strings.

Args:
addresses (list[IPvxAddress]): list of IP addresses in ipaddress format

Returns:
list[str]: list of IP strings

static cidr_exclude(*cidr1: str*, *cidrs2: list[str]*) → list[str]

Exclude cidr1 from any of networks in cidrs2.

Args:
cidr1 (str): cidr to be excluded.

cidrs2 (list[str]): list fo cidrs from which cidr1 will be excluded.

Returns:
list[str]: Resulting list of cidrs ("cidrs2" - "cidr1")

static cidr_iptype(*address: Any*) → str

Determines if address string is valid ipv4 or ipv6 or not.

Args:

address (Any):
address or cidr string

Returns:

str :
'ip4' or 'ip6' or empty string, '', if address invalid. Note. Earlier versions returned None instead of empty string if unable to be converted.

static cidr_is_subnet(*cidr: str*, *ipa_nets: list[py_cidr._network.cidr_types.IPvxNetwork]*) → bool

Check if cidr is a subnet of any of the list of IPvxNetworks .

Args:
cidr (str): Cidr string to check.

ipa_nets (list[IPvxNetwork]): list of IPvxNetworks to check.

Returns:
bool: True if cidr is subnet of any of the ipa_nets, else False.

static cidr_list_compact(*cidrs: list[str]*, *string: bool = True*) → list[str] | list[py_cidr._network.cidr_types.IPvxNetwork]

Compact list of cidr networks to smallest list possible. Deprecated - use compact_cidrs(cidrs, return_nets)) instead, it is the same with the boolean flag reversed.

Args:
cidrs (list[str]): list of cidr strings to compact.

string (bool):

- If True (default), then return is a list of strings.
- If False, a list of IPvxNetworks.

Returns:
list[str] | list[IPvxNetwork]: Compressed list of cidrs as ipaddress networks (string=False) or list of strings when string=True

static cidr_range_split(*cidr: str*) → tuple[str, str, str]

Convert network to IP Range.

Args:
net (IPvxNetwork): The network (IPvxNetwork) to examine.

string (bool): If True then returns cidr strings instead of IPvxAddress

Returns:
tuple[str, str, str]: that are strings of the first IP, middle IP and last IP in the cidr block

static cidr_set_prefix(*cidr: str*, *prefix: int*) → str

Set new prefix for cidr and return new cidr string.

Args:
cidr (str): Cidr string to use

prefix (int): The new prefix to use

Returns:
str: Cidr string using the specified prefix

static cidr_to_net(*cidr: str*, *strict: bool = False*) → py_cidr._network.cidr_types.IPvxNetwork | None

Convert cidr string to ipaddress network.

Args:
cidr (str): Input cidr string

strict (bool): If true then cidr is considered invalid if host bits are set. Defaults to False. (see ipaddress docs).

Returns:
IPvxNetwork | None: The ipaddress network derived from cidr string as IPvxNetwork = IPv4Network or IPv6Network or None if invalid.

static cidr_to_range(*cidr: str*, *string: bool = False*) → tuple[py_cidr._network.cidr_types.IPvxAddress | str | None, py_cidr._network.cidr_types.IPvxAddress | str | None]

Cidr string to an IP Range.

Args:
cidr (str): The cidr string to examine.

string (bool): If True then returns cidr strings instead of IPvxAddress

Returns:
tuple[IPAddress, IPAddress]: tuple (ip0, ip1) of first and last IP address in net (ip0, ip1) are IPvxAddress or str when string is True

static cidr_type_network(*cidr: str*) → tuple[str, type[py_cidr._network.cidr_types.IPvxNetwork]]

Cidr Network Type.

Args:
cidr (str): Cidr string to examine

Returns:
tuple[str, IPvxNetwork]: tuple(ip-type, net-type). ip-type is a string ('ip4', 'ip6') while network type is IPv4Network or IPv6Network

static cidrs2_minus_cidrs1(*cidrs1: list[str]*, *cidrs2: list[str]*) → list[str]

Exclude all of cidrs1 from cidrs2.

i.e. return "cidrs2" - "cidrs1".

Args:
cidrs1 (list[str]): list of cidr strings to be excluded.

cidrs2 (list[str]): list of cidr strings from which cidrs1 are excluded.

Returns:
list[str]: Resulting list of cidr strings = "cidrs2" - "cidrs1".

static cidrs_exclude(*cidrs1: list[str]*, *cidrs2: list[str]*) → list[str]

Deprecated: replaced by cidrs2_minus_cidrs1()

static cidrs_split_type(*cidrs: list[str]*) → tuple[list[str], list[str], list[str]]

Split a list of cidrs into ipv4, ipv6 and other.

Args:

cidrs (list[str]):
list of cidr strings

Returns:
tuple[ip4: list[str], ip6: list[str], other: list[str]] Tuple of lists of ipv4, ipv6 and unknown.

static cidrs_to_nets(*cidrs: list[str]*, *strict: bool = False*) →
list[py_cidr._network.cidr_types.IPvxNetwork]

Convert list of cidr strings to list of IPvxNetwork.

Args:
cidrs (list[str]): list of cidr strings

strict (bool): If true, cidr with host bits set is invalid. Defaults to false.

Returns:
list[IPvxNetwork]: list of IPvxNetworks generated from cidrs.

static clean_cidr(*cidr: str*) → str | None

Clean up a cidr address.

Does:

- fix up host bits to match the prefix
- convert old class A,B,C style IPv4 addresses to cidr.

e.g.
a.b.c -> a.b.c.0/24 a.b.c.23/24 -> a.b.c.0/24

Args:
cidr (str): Cidr string to clean up.

Returns:

str | None:

- cidr string if valid
- None if cidr is invalid.

static clean_cidrs(*cidrs: list[str]*) → list[str]

Clean list of cidrs.

Similar to clean_cidr() but for a list.

Args:
cidrs (list[str]): list of cidr strings to clean up.

Returns:
list[str]: list of cleaned cidrs. If input cidr is invalid then its returnded as None

static compact(*cidrs: list[str]*) → list[str]

Compact list of cidrs - can be mixed ipv4/ipv6. Returns 1 type, making type annotation simpler for caller.

Args:

cidrs (list[str]):
Input list of cidr strings.

Returns:

list[str]:
Compacted list of cidrs. If mixed ipv4 is before ipv6

Recommended methods are compact() or compact_nets(). Others are kept for backward compatibility.

static compact_cidrs(*cidrs: list[str]*, *nets: bool = False*) → list[str] | list[py_cidr._network.cidr_types.IPvxNetwork]

Compact a list of cidr networks as strings.

Args:
cidrs (list[str]): list of cidrs to compact.

nets (bool): If False, the default, the result will be list of strings else a list of IPvxNetwork's.

Returns:
list[str | IPvxNetwork]: A list of compacted networks whose elements are strings if return_nets is False or IPvxNetworks if True.

static compact_cidrs_to_cidrs(*cidrs: list[str]*) → list[str]

Compact list of cidr networks and return list of cidr strings

Same as compact_cidrs(cidrs, nets=False). With single return type this is helpful when using type annotation checkers.

static compact_nets(*nets: list[py_cidr._network.cidr_types.IPvxNetwork]*) → list[py_cidr._network.cidr_types.IPvxNetwork]

Compact list of IPvxNetwork.

Args:
nets (list[IPvxNetwork]): Input list if networks to compact.

Returns:
list[IPvxNetwork]: Compacted list of IPvxNetworks.

static fix_cidr_host_bits(*cidr: str*, *verb: bool = False*) → str

zero out any host bits.

A strictly valid cidr address must have host bits set to zero.

Args:
cidr (str): The cidr to "fix" if needed.

verb (bool): Some info on stdout when set True. Defaults to False.

Returns:
str: The cidr with any non-zero host bits now zeroed out.

static fix_cidrs_host_bits(*cidrs: list[str]*, *verb: bool = False*) → list[str]

zero any host bits for a list of cidrs.

Similar to fix_cidr_host_bits() but for a list of cidrs.

Args:
cidrs (list[str]): list of cidrs to fix up.

verb (bool): Some info on stdout when set True. Defaults to False.

Returns:
list[str]: The list of cidrs each with any non-zero host bits now zeroed out.

static get_host_bits(*ip: str*, *pfx: int = 24*) → int

Gets the host bits from an IP address given the netmask.

Args:
ip (str): The IP to examine.

pfx (int): The cidr prefix.

Returns:
int: The host bits from the IP.

static ip_to_address(*ip: str*) → py_cidr._network.cidr_types.IPvxAddress | None

Return ipaddress of given ip.

If IP has prefix or host bits set, strip the prefix and keep host bits.

Args:
ip (str): The IP string to convert

Rreturns (IPvxAddress | None): IPvxAddress derived from IP or None if not an IP address.

static ip_version(*addr_or_net: str | py_cidr._network.cidr_types.IPvxNetwork | py_cidr._network.cidr_types.IPvxAddress*) → int

Determines the IP version of an address or network:

Args:

addr_or_net (str | IPvxNetwork | IPvxAddress):
Can be IP addess or network either a string or ip_address format.

Returns:

int:
4 if addr_or_net is IPv4Network or IPv4Address 6 if addr_or_net is IPv6Network or IPv5Address 0 otherwise

static ipaddr_cidr_from_string(*address: str*, *strict: bool = False*) → py_cidr._network.cidr_types.IPvxNetwork | None

Convert string of IP address or cidr net to IPvxNetwork

Args:
address: IP or CIDR network as a string.

strict (bool): If true, host bits are disallowed for cidr block.

Returns:
IPvxNetwork | None: An IPvxNetwork or None if invalid.

static ips_to_addresses(*ips: list[str]*) → list[py_cidr._network.cidr_types.IPvxAddress]

Convert list of IP strings to a list of ip addresses

Args:
ips (list[str]): list of IP strings to convert

Returns:
list[IPvxAddress]: list of IPvxAddress derived from input IPs.

static is_rfc_1918(*cidr: str*) → bool

Check if cidr is any RFC 1918.

Args:
cidr (str): IP or Cidr to check if RFC 1918.

Returns:
bool: True if cidr is an RFC 1918 address. False if not.

static is_valid_cidr(*address: Any*) → bool

Check if address is a valid ip or cidr network.

Args:
address (Any): Address to check. Host bits set is permitted for a cidr network.

Returns:
bool: True/False if address is valid IPv4 or IPv6 address or network.

static is_valid_ip4(*address: Any*) → bool

check if valid IPv4 address or cidr.

Args:
address (Any): Check if this is a valid IPv4 address or cidr.

Returns:
bool: True if valid IPv4 else False

static is_valid_ip6(*address: Any*) → bool

check if valid IPv6 address or cidr.

Args:
address (Any): Check if this is a valid IPv6 address or cidr.

Returns:
bool: True if valid IPv6 else False

static net_exclude(*net1: py_cidr._network.cidr_types.IPvxNetwork*, *nets2: list[py_cidr._network.cidr_types.IPvxNetwork]*) → list[py_cidr._network.cidr_types.IPvxNetwork]

Exclude net1 from any of networks in net2 and return resulting list of nets (without net1).

Args:

net1 (IPvxNetwork): Network to be ecluded.

nets2 (list[IPvxNetwork]): list of networks from which net1 will be excluded from.

Returns:

list[IPvxNetwork]: Resultant list of networks "nets2 - net1".

static net_is_subnet(*net1: py_cidr._network.cidr_types.IPvxNetwork*, *net2: py_cidr._network.cidr_types.IPvxNetwork | list[py_cidr._network.cidr_types.IPvxNetwork]*) → bool

Determines if net1 is a subnet of any of net2.

Args:

net1 (IPvxNetwork):
Network to check if is a subnet.

net2 (IPvxNetwork | list[IPvxNetwork]):
Network or list of networks to be checked.

Returns:

bool:
True if net1 is a subnet of any of net2.

static net_range_split(*net: py_cidr._network.cidr_types.IPvxNetwork*) → tuple[py_cidr._network.cidr_types.IPvxAddress, py_cidr._network.cidr_types.IPvxAddress, py_cidr._network.cidr_types.IPvxAddress]

Convert network to IP Range.

Args:

net (IPvxNetwork): The network (IPvxNetwork) to examine.

string (bool): If True then returns cidr strings instead of IPvxAddress

Returns:

tuple[, IPvxAddressIPAddress, , IPvxAddressIPAddress, IPvxAddress]: that are the first IP, middle IP and last IP in the cidr block

static net_to_cidr(*net: py_cidr._network.cidr_types.IPvxNetwork*) → str

Net to Cidr String
Convert an ipaddress network to a cidr string.

Args:

net (IPvxNetwork):
Ipaddress Network to convert.

Returns:

str: Cidr string from net. If unable to conver, then empty string is returned.

static net_to_range(*net: py_cidr._network.cidr_types.IPvxNetwork*, *string: bool = False*) → tuple[py_cidr._network.cidr_types.IPvxAddress | str | None, py_cidr._network.cidr_types.IPvxAddress | str | None]

Convert network to IP Range.

Args:
net (IPvxNetwork): The network (IPvxNetwork) to examine.

string (bool): If True then returns cidr strings instead of IPvxAddress

Returns:
tuple[IPAddress, IPAddress]: tuple (ip0, ip1) of first and last IP address in net Each (ip0, ip1) is IPvxAddress or a string if "string" == True

static nets_exclude(*nets1: list[py_cidr._network.cidr_types.IPvxNetwork]*, *nets2: list[py_cidr._network.cidr_types.IPvxNetwork]*) → list[py_cidr._network.cidr_types.IPvxNetwork]

Exclude every nets1 network from from any networks in nets2.

Similar to net_exclude() except this version has a list to be excluded instead of a single network.

Args:
nets1 (list[IPvxNetwork]): list of nets to be excluded.

nets2: (list[IPvxNetwork]): list of nets from which will exclude any of nets1.

Returns:
list[IPvxNetwork]: list of resultant networks ("nets2" - "nets1")

static nets_to_cidrs(*nets: list[py_cidr._network.cidr_types.IPvxNetwork]*) → list[str]

Convert list of ipaddress networks to list of cidr strings.

Args:
nets (list[IPvxNetwork]): list of nets to convert.

Returns:
list[str]: list of cidr strings.

static range_to_cidrs(*addr_start: py_cidr._network.cidr_types.IPAddress*, *addr_end: py_cidr._network.cidr_types.IPAddress*, *string: bool = False*) → list[py_cidr._network.cidr_types.IPvxNetwork] | list[str]

Generate a list of cidr/nets from an IP range.

Args:
addr_start (IPAddress): Start of IP range

addr_end (IPAddress): End of IP range

string (bool): If True then returns list of cidr strings otherwise IPvxNetwork

Returns:
list[IPvxNetwork] | list[str] list of cidr network blocks representing the IP range. list elements are IPvxAddress or str if parameter string=True

static remove_rfc_1918(*cidrs_in: str | list[str]*) → tuple[str | list[str], str | list[str]]

Given list of cidrs, return list without any rfc 1918

Args:
cidrs_in (str | list[str]: Cidr string or list of cidr strings.

Returns:
tuple[str | list[str], str | list[str]]: Returns (tuple[cidrs_cleaned, rfc_1918_cidrs_found]):

- cidrs_cleaned: list of cidrs with all rfc_1918 removed.

- rfc_1918_cidrs_found: list of any rfc 1918 found in the input.

If input cidr(s) is a list, then items in output are a (possibly empty) list If not a list then returned items will be string or None.

static rfc_1918_cidrs() → list[str]

Return list of rfc 1918 networks cidr strings

Returns (list[str]):
list of RFC 1918 networks as cidr strings

static rfc_1918_nets() → list[py_cidr._network.cidr_types.IPvxNetwork]

Return list of rfc 1918 networks

Returns:
list[IPv4Network]: list of RFC 1918 networks.

static sort_cidrs(*cidrs: list[str]*) → list[str]

Sort the list of cidr strings.

Args:
cidrs (list[str]): list of cidrs.

Returns:
list[str]: Sorted copy of cidr list

static sort_ips(*ips: list[str]*) → list[str]

Sort a list of IP addresses.

Args:
ips (list[str]): list of ips to be sorted.

Returns:
list[str]: Sorted copy of ips.

static sort_nets(*nets: list[py_cidr._network.cidr_types.IPvxNetwork]*) → list[py_cidr._network.cidr_types.IPvxNetwork]

Sort a list of networks.

Args:

nets (list[IPvxNetwork]):
list of networks to be sorted.

Returns:

list[IPvxNetwork]:
Sorted copy of networks.

static version() → str

Returns
Version of py-cidr

py_cidr.cidr_file_class

CidrFile class: Read/write a file with list of cidr blocks as strings For reading

- comments ignored
- pname = is path to the file.
- cidr are all in column 1

Module Contents

class CidrFile

Provides common CIDR string file reader/writer tools.

All methods are static so no class instance variable needed.

static copy_cidr_file(*src_file: str*, *dst_file: str*) → bool

Copy one file to another.

Args:
src_file (str): Source file to copy.

dst_file (str): Where to save copy

Returns:
bool: True if all okay else False

static read_cidr_file(*fname: str*, *verb: bool = False*) → list[str]

Read file of cidrs and return list of all IPv4 and IPv6.

See read_cidrs() which this uses.

Args:
fname (str): Path to file of cidrs to read.

verb (bool): More verbose output

Returns:
list[str]: list of all cidrs (ip4 and ip6 combined)

static read_cidr_files(*targ_dir: str*, *file_list: list[str]*) → list[str]

Read files in a directory and return merged list of cidr strings.

Args:
targ_dir (str): Directory to find each file.

file_list (list[str]): list of files in *targ_dir* to read.

Returns:
list[str]: list of all cidrs found in the files.

static read_cidrs(*fname: str | None*, *verb: bool = False*) → tuple[list[str], list[str]]

Read file of cidrs and return tuple of separate lists (ip4, ip6).

- if fname is None or sys.stdin then data is read from stdin.
- only column 1 of file is used.
- comments are ignored

Args:
fname (str | None): File name to read.

verb (bool): More verbose output when True.

Returns:
tuple[list[str], list[str]]: tuple of lists of cidrs (ip4, ip6)

static write_cidr_file(*cidrs: list[str]*, *pname: str*) → bool

Write list of cidrs to a file.

Args:
cidrs (list[str]): list of cidr strings to write.

pname (str): Path to file where cidrs are to be written.

Returns:
bool: True if successful otherwise False.

py_cidr.cidr_map

Map cidr prefixes to some value.

Use separate maps for ipv4 and ipv6

Module Contents

class CidrMap(*cache_dir: str = ''*, *compact: bool = False*)

Class provides map(cidr) -> some value.

- ipv4 and ipv6 are cached separately
- built on CidrCache and Cidr classes

Args:
cache_dir (str): Optional directory to save cache file

todo: generalize value to be any object not just string # def __init__(self, cache_dir: str | None = None):

add_cidr(*cidr: str*, *val: Any*, *priv_maps: py_cidr._prefix.PrefixMaps | None = None*)

Historical version of add_prefix_val() - use that instead please.

add_cidrs(*prefixes: list[str]*, *vals: list[Any]*)

Historical version - use add_prefix_vals() instead.

add_prefix_val(*prefix_val: py_cidr._network.PrefixVal*, *priv_maps: py_cidr._prefix.PrefixMaps | None = None*)

Add cidr to cache.

Args:

prefix_val (PrefixVal):
PrefixVal = tuple[prefix: str, val: Any] Add this (prefix, val) pair to the map.

priv_map (private):
If using multiple processes/threads then provide this object where changes are kept instead of in the instance cache. This way the same instance (and its cache) can be used across multiple processes/threads.

Use CidrMap.create_private_cache() to create private_data

add_prefix_vals(*prefix_vals: list[py_cidr._network.PrefixVal]*)

Add list if (prefix, val) tuples.

Args:

prefix_vals (list[PrefixVal]):
List of tuples each being (prefix: str, val: Any)

static create_private_cache() → py_cidr._prefix.PrefixMaps

Create and Return private cache object to use with add_cidr().

This cache has no cache_dir set - memory only. Required if one CidrMap instance is used in multiple processes/threads Give each process/thread a private data cache and they can be merged into the CidrMap instance after they have all completed.

Returns:
(private): private_cache_data object.

items(*v6: bool = False*) → Iterator[tuple[str, Any]]

Iterator that returns oen element at a time of the map. Args:

v6 (bool):
Default is false, and the elements are from IPv4 map If True, then elements are taken from the IPv6 map.

Returns:

tuple[cidr: str, value: str]:
The (cidr, value) for each map element

lookup(*cidr: str*) → Any | None

Deprecated: Historical. Change to lookup_lmp()

Same as lookup_lmp() but only returns value not (prefix, value).

Returns the value of associated with lmp prefix for which cidr is subnet (or same) as. The prefix returned is the longest matching prefix (LMP).

Similar to lookup_both() but only the value is returned instead of both (prefix, value).

Args:

cidr (str):
Cidr value to lookup.

Returns:

Any | None:
Result = map(cidr) if found else None.

lookup_all(*cidr: str*) → list[tuple[str, Any]]

If cidr is in the map, return list of all (prefix, val) tuples. cidr is a subnet of each prefix. The first (prefix, val) returned is always the longest matching prefix (LMP) and it's value: (lmp, val)

The remaining elements will all have shorter prefix length (larger, less specific) network blocks.

Args:

cidr (str):
Cidr value to lookup.

Returns:

list[tuple[prefix: str, val: Any]]
Result = map(cidr). If no matching elements found returns empty list. Otherwise returns list of values for each element for which cidr is a subnet.

lookup_both(*cidr: str*) → tuple[str, Any]

Deprecated: Historical - same as lookup_lmp()

lookup_lmp(*cidr: str*) → tuple[str, Any]

Check if cidr is in the map. Similar to lookup() but returns the longest matching prefix (LMP) and it's value. cidr is then the same as or subnet of prefix. If not found then empty string for prefix.

See lookup_all() which returns list of (prefix, val) tuples where the first element is the (lmp, val) pair.

Args:

cidr (str):
Cidr value to lookup.

Returns:

tuple[prefix: str, value: Any]
cidr is same as or subnet of prefix and val it's assocated value. If not found then prefix is empty string.

merge(*priv_maps: py_cidr._prefix.PrefixMaps | None*)

Merge private maps back into into our own maps.

Args:

priv_maps (PrefixMaps):
The "private data" to add map. Merge the content of priv_maps into the current data. See CidrMap.create_private_cache()

print()

Print the cache data.

save_cache()

Write cache to files

PYTHON MODULE INDEX

p

INDEX

R

S

V

W